

Streaming Translation Pipeline

On-demand, real-time translation triggered by playback. When a user presses play on an untranslated audiobook, the system dispatches **30-second segments** to GPU workers, buffers three minutes of translated audio, then begins playback. Pre-translated books serve instantly from cache. (Chapter-at-a-time work is the batch pipeline; streaming is segment-granular.)

Not operational in the reference deployment

This pipeline requires an STT backend that the project does not provide. The maintainer's RunPod account is **decommissioned** and Vast.ai was never enabled, so no STT endpoint is configured. `library/localization/pipeline.py` raises `RuntimeError("No STT provider configured")` when asked to transcribe.

Everything below describes the code as written, and the code is unchanged. But **as deployed, nothing is dispatched and nothing completes**. The three systemd units are enabled and running; the sampler tier logs the truth every 5 minutes:

```
SAMPLER STALE: no sampler_job has completed in N days (threshold 3).
The timer is running; the work is not.
```

Already-cached chapters continue to serve normally — the cache is unaffected. To turn the subsystem off, see “Turning audio translation off” in `docs/MULTI-LANGUAGE-SETUP.md` (note that `upgrade.sh` re-enables the units on every run, so `systemctl disable` alone is not durable).

Why Streaming Exists

Batch-translating an entire library upfront (STT + DeepL + TTS for every chapter in every locale) would cost hundreds of dollars in GPU time, so the pipeline is built to pay only for what a listener actually plays.

(Earlier revisions quoted “1,861 audiobooks / 327 pre-translated / 5,245 chapters / 1,534 remaining” as of v8.3.0. Those are one deployment's figures at one point in time and have since drifted; they are omitted rather than maintained. Note also that a large share of existing `chapter_subtitles` rows are stamped `stt_provider='vastai-whisper'` — historical output from a provider whose client code was removed in v8.3.10.6.)

Streaming solves this by paying only for what a listener actually plays.

Two Pipelines, One Cache

Pipeline	Trigger	Processing	Output
Batch (<code>batch-translate.py</code>)	Operator-run + queue	Entire chapters, background	Permanent VTT + TTS audio
Streaming (<code>streaming-translate.py</code>)	Playback	30-second segments, real-time	Segments → consolidated VTT

Both pipelines write to the same permanent cache (`chapter_subtitles` and `chapter_translations_audio` tables). Once a chapter is translated by either pipeline, future plays are free. By design the cache self-heals: listening patterns gradually fill it, and batch fills the rest.

Two caveats on that design. `scripts/batch-translate.py` is **operator-run only** — no systemd unit or timer invokes it, so “batch fills the rest during idle time” describes an intent, not a mechanism. And in the reference deployment neither pipeline runs at all, for want of an STT backend.

End-to-End Playback Flow

Phase 1 — App Open (GPU Warm-Up)

When the app opens and the user’s locale is not English, the frontend sends `POST /api/translate/warmup`. This writes a row to `streaming_sessions` (`audiobook_id=0`, `locale='warmup'`).

Nothing consumes that row today. `grep warmup scripts/stream-translate-worker.py` returns no matches, and the handler says as much itself — “the actual GPU warm-up will be handled by the translation daemon when it sees this signal” (`streaming_translate.py:1384-1404`). There is no such daemon. No priming request is dispatched and no connectivity is verified; the endpoint is a no-op placeholder.

See `docs/SERVERLESS-OPS.md` for the `streaming/backlog` endpoint split and `docs/TRANSLATION-MONITOR.md` for the stuck-claim reset contract (live 60 s, sampler 2 h). Earlier revisions of this section referenced a “dual-provider D+C topology”, a 15-minute warmup expiry, and a 10-minute worker reclaim — Vast.ai was removed in v8.3.10.6, and neither of those two timers exists anywhere in the code.

Phase 2 — Press Play

`shell.js` calls `streamingTranslate.check(bookId, locale)`, which sends `POST /api/translate/stream` to the coordinator:

```
Player → Coordinator API → Database lookup:
├── chapter_subtitles exists? (batch cache)
├── chapter_translations_audio exists? (batch TTS cache)
├── Both exist → { state: "cached" } → instant playback
└── Missing → { state: "buffering", session_id, segment_bitmap }
```

Phase 3 — Buffering

The frontend state machine transitions from IDLE to BUFFERING:

1. A **visual overlay** slides up above the player bar — gold-themed progress bar showing segment completion (e.g., “3 / 6”)
2. A **localized audio notification** plays via pre-generated edge-tts clips (e.g., zh-Hans: “请稍候，正在为您翻译本书。”)
3. The **main audio pauses** — no point playing English narration during the wait

The coordinator simultaneously:

- Creates `streaming_segments` rows for the **entire active chapter** at **P0**
- Creates rows for the **entire next chapter** at **P1** (prefetch)
- Each row represents one 30-second segment: (`audiobook_id`, `chapter_index`, `segment_index`, `locale`, `state='pending'`)

Both calls go through `_ensure_chapter_segments(...)`, whose contract is “ensure segment rows exist for the entire chapter at the requested priority” (`streaming_translate.py:475-476`, `1062-1069`).

The six-segment cursor window is a seek-time construct, not a press-play one. BUFFER_AHEAD_SEGMENTS = 6 (≈ 3 minutes) is applied only in handle_seek_impl (:837-857). On press-play the whole chapter is queued and the worker's claim order determines what actually gets translated first.

See Priority Model below for the full four-tier semantics.

Phase 4 — GPU Worker Processing

stream-translate-worker.py polls the streaming_segments table in priority order and processes each segment:

1. Atomically claim next pending segment
(ORDER BY priority, active-chapter-first, chapter, segment)
2. ffmpeg stream-copy → extract 30-second audio slice from the chapter
3. STT (faster-whisper on GPU) → raw English transcript
4. Translation (DeepL API) → translated text
5. Generate VTT with timestamps
6. Offset timestamps for segment position within the chapter
7. POST /api/translate/segment-complete → report inline VTT content

P0 cursor-buffer segments are processed first so playback can resume as quickly as possible. Once the 3-minute buffer is satisfied, workers drain P1 (forward chase) to stay ahead of the cursor, and only then P2 (back-fill) to complete the timeline behind the cursor for the side panel and future backward scrubs.

Phase 5 — Real-Time Push

When the coordinator receives a segment completion callback, it:

1. Updates the segment state to completed in the database
2. Broadcasts segment_ready via WebSocket to all connected clients
3. Broadcasts buffer_progress with completed/total counts

The frontend receives these events and updates the progress bar in real time.

Phase 6 — Buffer Threshold Met

Once 6 segments are complete (3 minutes of audio), the state machine transitions from BUFFERING to STREAMING:

- Overlay hides
- Notification audio stops
- Main audio **resumes** with translated subtitles available
- GPU workers continue processing remaining segments ahead of the cursor

Phase 7 — Seek Handling

Action	Behavior
± 30 seconds within buffer	Instant — segment already cached, no interruption
Jump beyond cached range	POST /api/translate/seek → reprioritize from new cursor → re-enter buffering
Jump to batch-cached chapter	Instant — already in permanent cache

On seek-beyond-buffer: all existing pending segments are downgraded to **P2**; the six segments forward of the new cursor are promoted or inserted at **P0** (cursor buffer fill); the remainder of the chapter past that buffer is queued at **P1** (forward chase); the gap between the prior translated tail and the new cursor is queued at **P2** (back-fill) so the side panel and any future backward scrub stay continuous.

On stop: every state='pending' row for (book, locale) is **DELETED** (stop_streaming_impl, streaming_translate.py:862-881). Segments already in processing are left alone — the worker finishes them and the segment-complete callback lands normally. Pre-v8.3.2 this demoted pending rows instead, but the worker drained p0/p1 and then chewed through the demoted rows, so Stop never really stopped. Resume re-creates rows from scratch. Historical note — back-fill preserves work for future resume and side-panel completeness rather than discarding the queue.

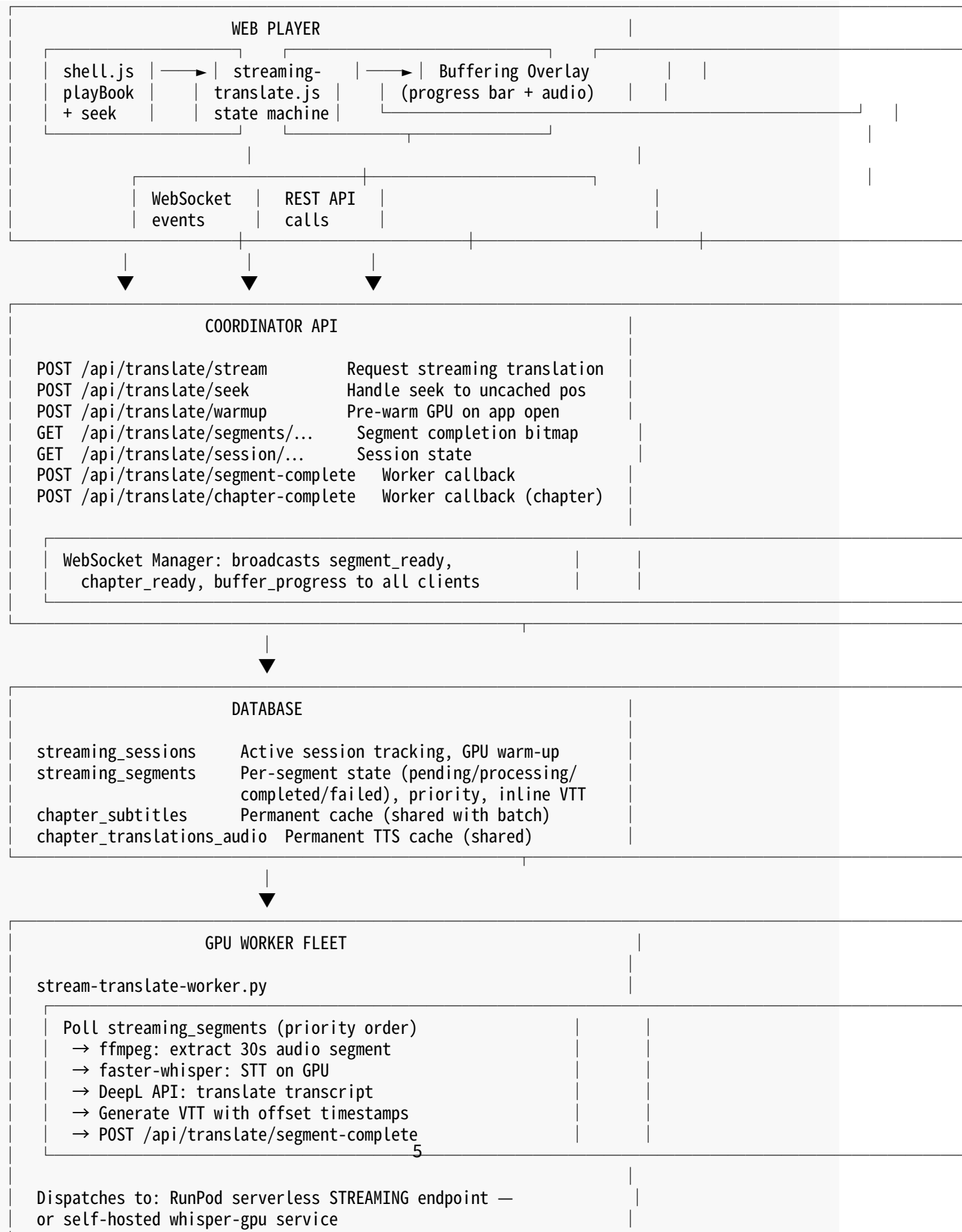
Phase 8 — Consolidation

When all segments of a chapter complete, `_consolidate_chapter()`:

1. Reads VTT content from all segment rows
2. Strips duplicate WEBVTT headers, merges into a single file
3. Writes to `${AUDIOBOOKS_VAR_DIR}/streaming-subtitles/{audiobook_id}/ch{NNN}.{locale}.vtt` (chapter index zero-padded to 3 digits, under the runtime var root — the install tree is read-only under `ProtectSystem=strict`)
4. Inserts into `chapter_subtitles` — the same permanent cache used by batch

After consolidation, the chapter is indistinguishable from a batch-translated one.

Architecture Diagram



Design Constants

Parameter	Value	Rationale
Segment duration	30 seconds	L40S processes in ~2-3s; small enough for low latency
Buffer threshold	6 segments (3 min)	Enough runway for continuous playback while GPU stays ahead
P0 — cursor buffer fill	6 segments forward of the cursor	Must flow before playback resumes
P1 — forward chase	Cursor buffer → end of chapter / next break	Keeps GPU ahead of the cursor during playback
P2 — back-fill	Prior translated tail → cursor	Continuous side panel and backward-scrub safety net

Priority Model (Cursor-Centric, Four Tiers since v8.3.8)

The scheduler is **cursor-centric**, not chapter-centric. Segments are queued at one of four priority tiers relative to the listener's current playback cursor. v8.3.8 added **p2=sampler** as a dedicated tier, pushing prior back-fill work to p3, so the 6-minute pretranslation sampler can never starve live playback:

Priority levels (lower = higher urgency):

- 0 P0 — cursor buffer fill. Populates first ~3 minutes (6 segments) forward of the cursor. Must flow before playback resumes. Live-playback, current book ONLY.
- 1 P1 — forward chase. Continues producing segments past the cursor buffer toward end-of-chapter / next logical break. Deprioritized only if the user jumps/stops. Live-playback, current book ONLY.
- 2 P2 — sampler (v8.3.8+). The 6-minute pretranslation for each book's opening. Runs for every book × enabled non-EN locale, bounded cost. See docs/SAMPLER.md. ENFORCED by DB trigger: an INSERT or UPDATE that would place an origin='sampler' row at priority<2 is ABORTed by the engine.
- 3 P3 — RESERVED, NOT IMPLEMENTED. No code path writes priority 3, and no row in production carries it. Back-fill currently shares priority 2 with the sampler and is distinguished only by `origin`. Described here as the intended shape. Would produce segments between prior translated tail and the cursor; runs after everything above is satisfied so the side panel and future backward-scrubbing have continuous context.

Per the trigger + invariant: live playback of the currently-playing book always preempts sampler work on any other book. The sampler can never pull a GPU slot away from a listener who's actively listening.

On seek-beyond-buffer: existing pending live segments are downgraded to ****P2**** (not P3 — `handle\_seek\_impl:838-843` writes 2); six segments forward of the new cursor promoted/inserted at P0; end-of-chapter remainder queued at P1; gap between prior tail and new cursor queued at P3.

On stop: all pending live segments are DELETED, not demoted (see above). Historical note — back-fill preserves work for future resume / side-panel completeness).

Worker claim order is ORDER BY s.priority ASC, CASE WHEN sess.active_chapter IS NOT NULL AND s.chapter_index = sess.active_chapter THEN 0 ELSE 1 END, s.chapter_index ASC, s.segment_index ASC (scripts/stream-translate-worker.py:336-341) — an active-chapter tiebreaker was added so a P0 row in chapter N+1 wins once the player crosses into N+1. The priority-first shape is otherwise unchanged; the tier set expanded in v8.3.8 to give the sampler its own protected slot.

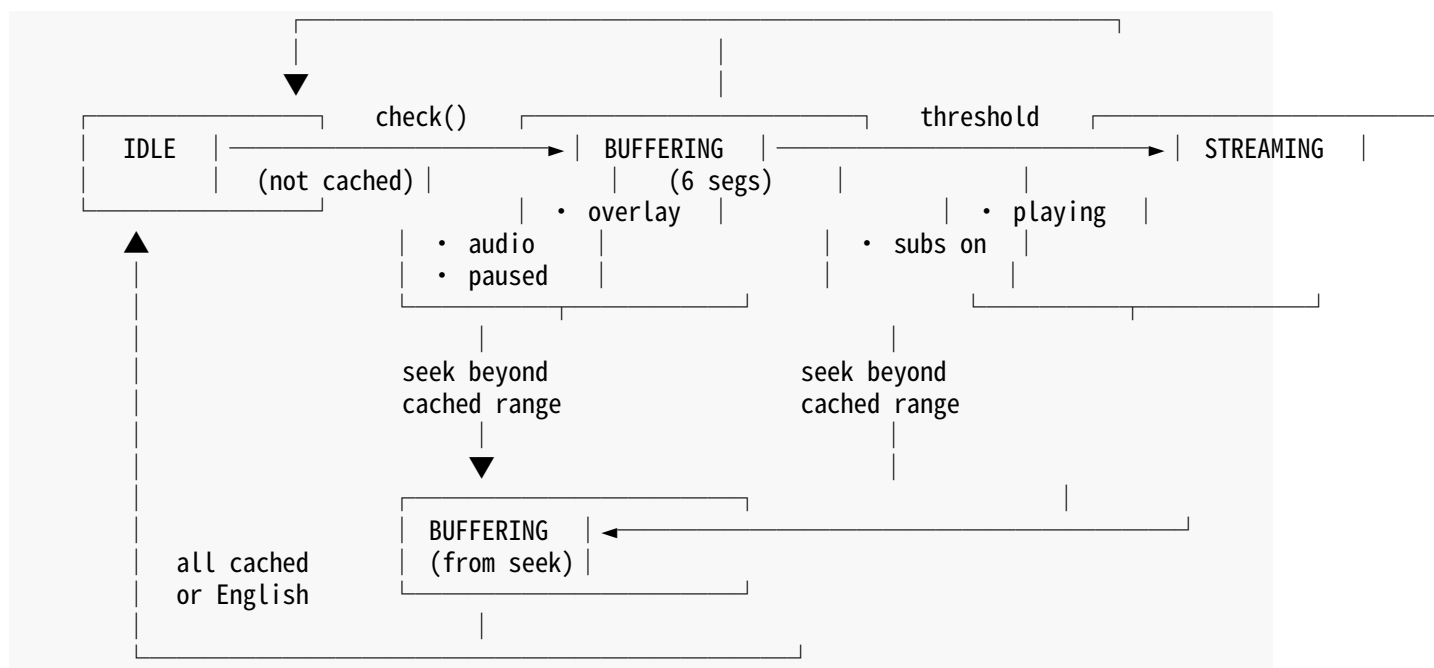
How the sampler interacts with this model

The sampler runs continuously at p2 as books are ingested, but it never competes with live playback (p0/p1). When a user plays a sample and crosses the **adaptive buffer-fill threshold** (segment 3 if no configured STT provider has ready workers, 4 if any provider is warm — see docs/SAMPLER.md § Adaptive buffer-fill threshold), the frontend calls POST /api/translate/sampler/activate, which creates p0/p1 segments from the cursor forward. GPU cold-start happens while the user is still listening to the cached sample; by the time the 6-minute sample ends, the live buffer is already filling ahead. Seamless transition, no spinner.

Transition Summary

Event	P0 (cursor buffer)	P1 (forward chase)	P2 (back-fill)
Press play	6 segments forward of cursor	Rest of current chapter	(empty)
Seek beyond buffer	6 segments forward of new cursor	Remainder after buffer	All prior pending + gap from prior tail to cursor
Stop	(empty)	(empty)	All pending segments
Resume	6 segments forward of cursor (re-promoted from P2)	Rest of chapter (re-promoted)	Prior tail → cursor remainder

State Machine



Controlling Batch Translation

The batch pipeline is independent and runs against the BACKLOG serverless endpoint pool (cold, min_workers=0). Dispatch happens inline from the API and via scripts/batch-translate.py, which reads translation_queue and processes pending rows chapter-at-a-time.

```
# Run a one-shot batch pass over pending queue rows
sudo -u audiobooks /opt/audiobooks/Library/venv/bin/python \
/opt/audiobooks/scripts/batch-translate.py
```

No GPU lifecycle to manage — serverless endpoints scale to zero on their own, so you are billed only for chapters actually translated. Idle cost is \$0 on BACKLOG pools.

Wedge detection: streaming_segments rows stuck in processing for more than 10 minutes are re-claimed by the streaming worker on its next poll. Batch-side stuck rows are reset to pending by the API reconcile loop.

Files

File	Purpose
library/backend/api_modular/streaming_candidates.py	Candidate API (14 endpoints)
library/web-v2/js/streaming-translate.js	Frontend state machine
library/web-v2/css/shell.css	Buffering overlay styles
library/web-v2/shell.html	Overlay markup
scripts/stream-translate-worker.py	Streaming GPU worker (segment processing, STREAMING endpoint pool)
scripts/stream-translate-daemon.sh	Long-running wrapper for the streaming worker
scripts/batch-translate.py	Batch worker (chapter processing, BACKLOG endpoint pool)
systemd/audiobook-stream-translate.service	Streaming worker service unit
library/localization/pipeline.py	Shared STT → translate → VTT pipeline (_remote_stt_candidates dispatches STREAMING vs BACKLOG)
library/web-v2/audio/translation-buffering-*.mp3	Localized notification clips

Database Schema (Migration 003)

```
CREATE TABLE streaming_sessions (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  audiobook_id INTEGER NOT NULL,
  locale      TEXT NOT NULL,
  state       TEXT DEFAULT 'buffering',    -- buffering, streaming, completed, warmup
  active_chapter INTEGER DEFAULT 0,
  buffer_threshold INTEGER DEFAULT 6,
  gpu_warm    INTEGER DEFAULT 0,
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE streaming_segments (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  audiobook_id INTEGER NOT NULL,
  chapter_index INTEGER NOT NULL,
  segment_index INTEGER NOT NULL,
  locale      TEXT NOT NULL,
  state       TEXT DEFAULT 'pending',    -- pending, processing, completed, failed
  priority    INTEGER NOT NULL DEFAULT 2, -- 0=P0 cursor buffer, 1=P1 forward chase, 2=P2 sampler/l
  worker_id   TEXT,
  vtt_content TEXT,                      -- translated-locale VTT for completed segments
  source_vtt_content TEXT,               -- source-language (English) VTT (v8.3.2+)
  audio_path  TEXT,                      -- per-segment opus, filled by worker (v8.3.2+)
  retry_count INTEGER DEFAULT 0,         -- transient failure recovery counter (v8.3.2+)
  started_at  DATETIME,
  completed_at DATETIME,
  UNIQUE(audiobook_id, chapter_index, segment_index, locale)
);
```

Schema evolved across several data-migrations — 003_streaming_segments.sh (MIN_VERSION=8.3.0,

creates both tables), 005_streaming_audio_webm.sh, 006_streaming_source_vtt.sh, 007_streaming_retry_count.sh, and 008_streaming_origin_and_sampler.sh (MIN_VERSION=8.3.8, adds the origin column, the sampler_jobs table, and the two RAISE(ABORT) priority triggers). All are idempotent (PRAGMA table_info guards) and boundary-gated via MIN_VERSION, so cross-version upgrades populate only what is missing.

The SQL above is abridged — it omits origin, error, created_at, the foreign keys, the three indexes, the sampler_jobs table, and the priority triggers. Read the migration scripts for the authoritative schema.

In-flight VTT Stitching (v8.3.7+)

The manifest and subtitle-fetch routes merge chapter_subtitles (finalized, on-disk VTT files) with a live index of streaming_segments rows so chapters whose VTT has not yet been consolidated still appear in the subtitle list the moment the first segment lands.

- `/api/audiobooks/<id>/subtitles` returns the union of (a) cached rows in chapter_subtitles and (b) a deduped (chapter_index, locale) index built from streaming_segments. Polling from subtitles.js discovers live-streaming tracks without waiting for end-of-chapter consolidation.
- `/api/audiobooks/<id>/subtitles/<chapter>/<locale>` falls through to a stitched VTT built from streaming_segments when no cached file exists on disk (or a row exists in chapter_subtitles but its file is missing). Stitching strips per-segment WEBVTT headers and emits a single WEBVTT + concatenated cues in segment_index order.
- For locale='en' the stitcher pulls source_vtt_content (the Whisper transcript is locale-agnostic); other locales pull vtt_content where streaming_segments.locale matches.
- Stitched VTT is **never cached on disk** — always rebuilt from segment rows so late-arriving segments appear on the next fetch.
- Error discrimination is preserved: a cached row with a missing on-disk file still returns VTT file missing on disk (404); no row at all returns Subtitle not found (404).

Deferred Legacy-Queue State (v8.3.7+)

Library/localization/queue.py::get_book_translation_status collapses pending / processing / failed rows on non-English locales to a new {"state": "deferred", "reason": "streaming_pipeline"} payload, masking pre-streaming-era batch-pipeline crashes from the UI. Before this change every first-open of an untranslated zh-Hans book rendered stale 字幕生成失败 — No STT provider configured toasts surfaced from translation_queue rows that had been failing since the legacy worker stopped draining months ago. The canonical progress surface for non-en locales is now the streaming overlay (implemented inside Library/web-v2/js/streaming-translate.js, with markup at shell.html's #streaming-overlay — there is no streaming-overlay.js file); completed legacy rows still pass through unchanged (legitimate VTT-on-disk cases). 'en' locale is exempt — STT failures for English are real, not stale.

Security

All route handlers validate inputs at the boundary:

- **Locale:** `_sanitize_locale()` enforces `^[a-zA-Z]{2}(:-?[a-zA-Z0-9]{1,8})?$` — rejects path traversal (..) and log injection (newlines, control characters)
- **Integer IDs:** audiobook_id, chapter_index, segment_index are coerced to int before any database query or filesystem operation
- **Worker callbacks** (segment-complete, chapter-complete): internal-only endpoints called by GPU workers, not exposed to browser clients